

TESTES DE SOFTWARE

Trabalho Prático de Avaliação

Plataforma GREENHERB — Gestão Inteligente de Estufa

Engenharia de Software II

Ano Letivo 2025/2026

Plano de Sprints

Sprint 1

- Criação de endpoints em Node.js.
- Criação de testes de unidade para a autenticação.
- Atualização do relatório com a matriz de rastreabilidade.

Sprint 2

- Criação de testes de unidade para a importação do catálogo de ervas aromáticas e criação de planos de cultivo.
- Atualização do relatório com a matriz de rastreabilidade.

<a completar semanalmente>

1. Introdução

O presente documento constitui o enunciado do trabalho prático da unidade curricular e serve, simultaneamente, de base ao relatório final a entregar pelos grupos de trabalho. O objetivo central consiste na conceção, implementação e execução de um plano de testes abrangente sobre a API REST que suporta a plataforma GREENHERB, descrita em detalhe no enunciado funcional fornecido em anexo.

A plataforma GREENHERB destina-se à gestão inteligente de uma estufa de ervas aromáticas, integrando funcionalidades de planeamento de cultivos, monitorização ambiental, geração de alertas, automação por regras e auditoria. Dada a sua natureza multi-perfil (Técnico, Responsável Técnico e Administrador) e a criticidade das regras de negócio, a qualidade da API REST que suporta o sistema deve ser rigorosamente avaliada.

Pretende-se que os grupos demonstrem domínio dos diferentes níveis de teste — unidade, integração e sistema — e apliquem técnicas formais de conceção de casos de teste, nomeadamente técnicas de caixa-preta (particionamento de equivalência e análise de valores limite) e técnicas de caixa-branca (cobertura de condições múltiplas).

2. Objetivos do Trabalho

No final do trabalho, os alunos deverão ser capazes de:

- Planear uma estratégia de testes documentada para uma API REST de complexidade média/elevada.
- Distinguir e implementar testes nos três níveis: unidade, integração e sistema.
- Aplicar técnicas de teste de caixa-preta — particionamento de equivalência e análise de valores limite — na derivação sistemática de casos de teste.
- Aplicar técnicas de teste de caixa-branca — cobertura de condições múltiplas (MC/DC e cobertura de condições combinadas) — sobre código produtivo.
- Medir e interpretar métricas de qualidade, nomeadamente cobertura de código, cobertura funcional e taxa de defeitos detetados.
- Documentar com rigor o processo, justificando opções, registando defeitos e propondo melhorias.

3. Sistema Sob Teste (SUT)

O Sistema Sob Teste (System Under Test) é a API REST que sustenta a aplicação GREENHERB. A API expõe, no mínimo, os recursos correspondentes às entidades do domínio descritas no enunciado funcional, conforme sintetizado de seguida.

3.1. Entidades e Recursos da API

Recurso	Descrição
/users	Gestão de utilizadores e perfis (Técnico, Responsável, Administrador).
/auth	Autenticação, emissão e renovação de tokens.
/herbs	Catálogo de ervas aromáticas, com suporte a importação CSV/Excel.
/plans	Planos de cultivo dos tipos regular, emergência e pontual.
/batches	Lotes de cultivo, com estado, divisões, perdas e produtividade.
/tasks	Tarefas operacionais (rega, fertilização, colheita, monitorização).
/measurements	Medições ambientais de temperatura, humidade e luminosidade.
/alerts	Alertas Informativos, Avisos e Críticos, com resolução ou ignorar justificado.
/automation	Regras de automação e comutação entre modo Manual e Automático.
/reports	Exportação de relatórios em CSV ou Excel.
/audit	Logs de auditoria de operações relevantes.

3.2. Premissas e Pressupostos

- A API segue a especificação OpenAPI 3.x e está documentada (Swagger/Redoc).
- A autenticação assenta em JWT, com expiração e renovação.
- A API valida os perfis de utilizador em cada endpoint protegido.
- A persistência é feita em base de dados relacional, mas os testes não devem depender de SGBD específico.
- A API expõe códigos HTTP normalizados (2xx, 4xx, 5xx) e respostas JSON estruturadas para erros.

4. Âmbito dos Testes

Os testes devem cobrir, no mínimo, as áreas funcionais e regras de negócio descritas no enunciado da plataforma GREENHERB. Em particular, deve ser dada especial atenção aos seguintes pontos críticos do domínio, escolhidos pela sua riqueza em condições, intervalos e regras combinadas:

- Validação de planos de cultivo (regular, emergência e pontual), incluindo intervalos de temperatura, humidade e luminosidade, e exigência de autorização explícita do Responsável Técnico para o plano pontual.
- Geração automática de alertas com base nas medições ambientais e nos limites definidos no plano associado ao lote.
- Classificação de alertas (Informativo, Aviso, Crítico) e regras de resolução/ignorar com justificação obrigatória.

- Transições de estado de lotes (ativo, concluído, comprometido), incluindo divisão parcial, registo de perdas e cálculo de produtividade.
- Comutação entre modo Manual e Automático nas regras de automação e suas consequências sobre tarefas executadas.
- Controlo de acesso por perfil de utilizador em cada endpoint relevante.
- Auditoria das operações: garantia de que toda a ação relevante é registada com utilizador, ação e momento.

5. Níveis de Teste a Implementar

Os grupos devem implementar, obrigatoriamente, testes nos três níveis seguintes. Para cada nível são indicados o foco, o tipo de artefactos esperados e exemplos representativos. A distribuição quantitativa exata é proposta na secção 8.

5.1. Testes de Unidade

Os testes de unidade visam validar, de forma isolada, o comportamento de funções, métodos e classes do código produtivo, recorrendo a duplos de teste (mocks, stubs, fakes) sempre que existam dependências externas, como base de dados, serviços ou ficheiros. O foco recai sobre a lógica de domínio.

Exemplos:

- Validadores de planos de cultivo (intervalos de temperatura/humidade/luminosidade, durações, dosagens).
- Classificador de alertas em função das medições e dos limites do plano.
- Calculadora de produtividade de lote (em função de perdas, divisões e tempo real vs. planeado).
- Motor de regras de automação que decide se uma ação é sugerida ou executada consoante o modo (Manual/Automático).
- Verificadores de coerência das medições recebidas (deteção de dados em falta ou incoerentes).

5.2. Testes de Integração

Os testes de integração validam a colaboração entre componentes — controladores, serviços, repositórios e base de dados real ou em memória — e o cumprimento dos contratos da API. Devem ser executados com a stack o mais próxima possível da produção, mas sem dependências externas voláteis.

Exemplos:

- Criar plano regular e associá-lo a um lote, garantindo persistência consistente em ambas as entidades.
- Submeter uma medição que viola limites do plano e verificar que é gerado um alerta com a classificação correta.

- Resolver um alerta como *Ignorado* sem justificação e confirmar que a API rejeita a operação.
- Importar ervas aromáticas a partir de um ficheiro CSV, validando linhas válidas, inválidas e parcialmente válidas.
- Exportar um relatório em CSV/Excel e validar a estrutura e o conteúdo do ficheiro produzido.

5.3. Testes de Sistema

Os testes de sistema avaliam a aplicação como um todo, executando fluxos end-to-end sobre a API em ambiente o mais realista possível. Recomenda-se a utilização de ferramentas como Postman/Newman, REST Assured, ou Supertest.

Exemplos:

1. Onboarding completo: criação de utilizadores nos três perfis pelo Administrador e respetivo login.
2. Ciclo completo de um lote: registo da erva, criação do plano regular, abertura do lote, execução de tarefas operacionais, registo de medições, eventual divisão de lote, fecho do lote e cálculo de produtividade.
3. Gestão de incidentes: medições fora dos limites do plano geram alertas; o Responsável Técnico cria um plano de emergência e este é aplicado ao lote.
4. Plano pontual: tentativa de execução sem autorização do Responsável Técnico (deve falhar) e tentativa com autorização (deve ser permitida).
5. Auditoria: para cada um dos fluxos anteriores, validar que as entradas correspondentes existem no log de auditoria com utilizador, ação e timestamp.

6. Técnicas de Conceção de Casos de Teste

A seleção de casos de teste não pode ser arbitrária. Os grupos devem aplicar, de forma sistemática, técnicas formais e justificar a sua aplicação no relatório.

6.1. Técnicas de Caixa-Preta (Black Box)

6.1.1. Particionamento de Equivalência

Para cada parâmetro de entrada relevante, devem ser identificadas classes de equivalência válidas e inválidas, e selecionado pelo menos um caso de teste representativo de cada classe. A técnica deve ser aplicada, no mínimo, aos seguintes parâmetros:

- Tipo de plano de cultivo: regular, emergência, pontual (válidos); qualquer outro valor (inválido).
- Classificação de alertas: Informativo, Aviso, Crítico (válidos); valores fora desta enumeração (inválidos).
- Estado de lote: ativo, concluído, comprometido (válidos); transições proibidas (inválidas).
- Perfil de utilizador: Técnico, Responsável, Administrador (válidos); ausência de token ou perfil desconhecido (inválidos).

- Decisão sobre alertas: Resolvido com justificação opcional; Ignorado com justificação obrigatória; outras combinações (inválidas).

6.1.2. Análise de Valores Limite

Para os parâmetros que envolvem intervalos numéricos ou contagens, devem ser testados os valores no limite inferior, imediatamente abaixo do limite inferior, no limite superior, imediatamente acima do limite superior e um valor nominal interior. A técnica deve ser aplicada, no mínimo, aos seguintes parâmetros:

Parâmetro	Intervalo Aceitável (exemplo)	Valores a Testar
Temperatura (°C)	[18, 28]	17, 18, 23, 28, 29
Humidade relativa (%)	[40, 80]	39, 40, 60, 80, 81
Luminosidade (lux)	[5000, 25000]	4999, 5000, 15000, 25000, 25001
Duração do ciclo (dias)	[1, 365]	0, 1, 90, 365, 366
Justificação para ignorar (chars)	[10, 500]	9, 10, 250, 500, 501

Nota: os intervalos exatos devem ser confirmados na documentação da API. A tabela apresenta valores plausíveis a título ilustrativo.

6.2. Técnicas de Caixa-Branca (White Box)

6.2.1. Cobertura de Condições Múltiplas

Sobre componentes de lógica não-trivial — tipicamente os referidos na secção 5.1 — devem ser aplicados critérios de cobertura mais fortes do que a simples cobertura de instruções ou de decisões. Em concreto, exige-se a aplicação de:

- Cobertura de condições múltiplas (Multiple Condition Coverage), em que devem ser exercitadas todas as combinações possíveis de valores lógicos das condições atómicas em cada decisão composta.
- Cobertura MC/DC (Modified Condition / Decision Coverage), em que cada condição atómica deve, isoladamente, demonstrar capacidade de afetar o resultado da decisão.

A técnica deve ser aplicada, no mínimo, aos seguintes pontos do código produtivo:

- Função de classificação de alertas, tipicamente com decisões compostas como *if (temperatura > limMaxT || humidade < limMinH) && sensorOK*.
- Função de validação de criação de plano pontual, que combina o tipo do plano, a presença de autorização do Responsável Técnico e a validade dos parâmetros.
- Função de transição de estado de lote, que depende de estado atual, existência de perdas registadas e datas reais.
- Função de decisão do motor de automação, que combina modo (Manual/Automático), regra ativa e medição recente.

6.2.2. Tabelas de Verdade e Justificação

Para cada decisão composta selecionada, o relatório deve apresentar:

6. A expressão lógica original, identificando as condições atômicas (C1, C2, ..., Cn).
7. A tabela de verdade completa (2^n linhas) ou a tabela MC/DC reduzida, conforme o critério aplicado.
8. O subconjunto mínimo de casos de teste que satisfaz o critério, com justificação.
9. A correspondência entre cada linha selecionada e o respetivo caso de teste implementado.

7. Ambiente, Ferramentas e Métricas

7.1. Ferramentas Recomendadas

A escolha das ferramentas é livre, desde que coerente com a stack tecnológica da API. A título indicativo:

- Testes de unidade e integração: JUnit 5, Mockito (Java); Jest, Mocha, Supertest (Node.js); pytest, unittest.mock (Python); xUnit, Moq (.NET).
- Testes de sistema/API: Postman + Newman, REST Assured, Karate, Cypress (com foco em API).
- Cobertura de código: JaCoCo, Istanbul/nyc, Coverage.py, Coverlet.
- Cobertura de condições e MC/DC: relatórios de JaCoCo (modo branch), PIT (mutation testing como complemento).

7.2. Métricas Mínimas a Reportar

- Cobertura de instruções (statement coverage).
- Cobertura de ramos / decisões (branch coverage).
- Cobertura de condições múltiplas / MC/DC nos componentes selecionados.
- Número de casos de teste por nível (unidade, integração, sistema).
- Tempo total de execução da bateria de testes.

8. Matriz de Rastreabilidade

A matriz de rastreabilidade é o instrumento que liga, de forma explícita e auditável, cada caso de teste aos requisitos funcionais, regras de negócio e endpoints da API que ele exercita. A sua manutenção é obrigatória ao longo do trabalho e o relatório final deve incluir a versão completa em anexo.

A matriz cumpre três objetivos fundamentais: (i) demonstrar que todos os requisitos relevantes possuem cobertura de teste, evitando lacunas; (ii) permitir avaliação rápida do impacto de uma alteração de requisito sobre o conjunto de testes; (iii) fundamentar quantitativamente a discussão de cobertura funcional no relatório.

8.1. Estrutura da Matriz

Cada linha da matriz representa um caso de teste e deve conter, no mínimo, as seguintes colunas:

- ID do Caso de Teste — identificador único, com prefixo indicativo do nível (TU = unidade, TI = integração, TS = sistema).
- Requisito / Regra de Negócio — referência ao requisito funcional ou regra do enunciado da plataforma GREENHERB.
- Endpoint(s) Exercitado(s) — método HTTP e caminho do recurso REST testado.
- Nível de Teste — Unidade, Integração ou Sistema.
- Técnica Aplicada — Particionamento de Equivalência, Valores Limite, Condições Múltiplas, ou combinação.
- Resultado Esperado — código HTTP esperado e/ou comportamento observável (e.g., alerta gerado, registro de auditoria criado).
- Pré-condições — estado do sistema que tem de existir antes da execução do teste: registros a inserir na base de dados, autenticação a obter, ficheiros a colocar no sistema, fixtures a carregar.

8.2. Exemplo de Matriz de Rastreabilidade

A tabela seguinte apresenta um excerto representativo da matriz de rastreabilidade, com casos de teste cobrindo os três níveis e as três técnicas exigidas. O excerto destina-se a ilustrar o formato e o nível de detalhe esperado; a matriz final entregue pelos grupos será necessariamente bastante mais extensa.

ID	Requisito / Regra	Endpoint	Nível	Técnica	Resultado Esperado	Pré-condições
TU-01	RN-01: intervalos do plano regular	(validador interno)	Unidade	Valores Limite (temp = 17, 18, 23, 28, 29 °C)	Validador rejeita 17 e 29; aceita 18, 23 e 28.	Nenhuma. Teste isolado sobre o validador, sem dependências de BD ou rede.
TU-02	RN-02: classificação de alertas	(classificador interno)	Unidade	Condições Múltiplas (temp > limMax, hum < limMin, sensorOK)	Tabela de verdade com 8 combinações; classificações Informativo/Aviso/Crítico corretas em todas.	Nenhuma. Limites do plano e leituras passados como parâmetros via mock.
TU-03	RN-08: cálculo de produtividade	(calculadora interna)	Unidade	Particionamento de Equivalência (com/sem perdas, com/sem divisão)	Produtividade calculada corretamente em todas as classes; lote sem fim real lança exceção.	Objetos Lote construídos em memória com diferentes combinações de perdas e divisões.

ID	Requisito / Regra	Endpoint	Nível	Técnica	Resultado Esperado	Pré-condições
TI-01	RF-04: criação de plano regular	POST /plans	Integração	Particionamento de Equivalência (tipo válido/inválido)	201 para tipos válidos (regular, emergência, pontual); 400 para tipo inválido.	Utilizador Responsável autenticado (JWT válido). Erva aromática previamente registada (FK do plano).
TI-02	RN-04: plano pontual exige autorização	POST /plans, POST /batches/{id}/apply-plan	Integração	Condições Múltiplas (tipo, autorização, parâmetros)	403 quando autorização ausente; 201 quando autorização presente e parâmetros válidos.	Lote ativo na BD; Técnico e Responsável criados; tokens JWT distintos para cada perfil.
TI-03	RF-07: registo de medições gera alerta	POST /measurements, GET /alerts	Integração	Valores Limite (humidade = 39, 40, 60, 80, 81 %)	Alerta gerado para 39 e 81; nenhum alerta para 40, 60, 80.	Lote ativo associado a plano regular com humidade [40, 80]. Tabela de alertas vazia no início do teste.
TI-04	RN-05: ignorar alerta exige justificação	PATCH /alerts/{id}	Integração	Valores Limite (justif. = 9, 10, 250, 500, 501 chars)	422 para 9 e 501 chars; 200 para 10, 250, 500 chars.	Alerta com estado pendente previamente inserido na BD; Responsável autenticado.
TI-05	RF-03: importação CSV de ervas	POST /herbs/import	Integração	Particionamento de Equivalência (linhas válidas/inválidas/parciais)	Resposta agregada com totais corretos por categoria; ficheiro vazio devolve 400.	Administrador autenticado; ficheiros fixture (válido, inválido, misto, vazio) disponíveis em /resources.
TI-06	RF-01: controlo de acesso por perfil	POST /users (qualquer perfil)	Integração	Particionamento de Equivalência (perfil = Téc/Resp/Admin/sem token)	201 apenas para Administrador; 403 para Técnico e Responsável; 401 sem token.	Três utilizadores em BD (um por perfil) com tokens válidos pré-emitidos. Cenário também executado sem header Authorization.
TS-01	Fluxo E2E: ciclo completo de lote	POST /herbs, /plans, /batches, /tasks, /measurements; GET /batches/{id}	Sistema	Combinação (PE + VL ao longo do fluxo)	Lote percorre estados ativo→concluído; produtividade calculada; auditoria completa.	BD em estado limpo (apenas seed mínimo: utilizadores dos três perfis). Tabelas de

ID	Requisito / Regra	Endpoint	Nível	Técnica	Resultado Esperado	Pré-condições
						domínio vazias.
TS-02	Fluxo E2E: gestão de incidente	POST /measurements, /plans (emergência), PATCH /alerts/{id}	Sistema	Condições Múltiplas + Valores Limite	Alerta Crítico gerado, plano de emergência aplicado, alerta resolvido com registro.	Lote ativo com plano regular já aplicado; Responsável autenticado; sensores em modo simulado.
TS-03	RN-09: auditoria de operações	GET /audit (após operações nos restantes endpoints)	Sistema	Particionamento de Equivalência (operações auditáveis vs. consultas)	Toda operação de escrita produz entrada com utilizador, ação e timestamp; consultas não.	Tabela de auditoria limpa antes do teste; lote e plano fixture pré-criados; Administrador autenticado para consultar /audit.

Notas sobre o exemplo apresentado:

- Os identificadores de requisito (RF-XX, RN-XX) referem-se à numeração que cada grupo deverá adotar ao decompor o enunciado funcional da plataforma GREENHERB em requisitos atômicos.
- As pré-condições devem ser materializadas em código através de fixtures, scripts SQL de seed, factories ou setup methods da framework de testes, garantindo que cada execução parte de um estado conhecido e reprodutível.
- Recomenda-se que a matriz seja mantida em formato tabular editável (folha de cálculo) durante o desenvolvimento e exportada para o relatório final.

8.3. Cobertura Bidirecional

A rastreabilidade deve ser bidirecional: para cada requisito é possível identificar quais os casos de teste que o cobrem, e para cada caso de teste é possível identificar qual o requisito que justifica a sua existência. O relatório final deve incluir, complementarmente, uma tabela inversa (Requisito → Casos de Teste) que evidencie a inexistência de requisitos sem cobertura.

Requisitos identificados sem qualquer caso de teste associado constituem lacunas e devem ser explicitamente assinalados, com justificação (e.g., requisito não funcional fora do âmbito, requisito dependente de funcionalidade não implementada).

9. Entregáveis e Estrutura do Relatório

O presente documento serve de base ao relatório final. Os grupos devem expandi-lo com as secções de execução e resultados. O relatório final deverá conter, no mínimo:

1. Identificação do grupo, repositório de código e instruções de execução.
2. Descrição do Sistema Sob Teste e do âmbito coberto.
3. Estratégia de testes adotada, com justificação.
4. Plano de testes detalhado, organizado pelos três níveis (unidade, integração, sistema).
5. Aplicação documentada de particionamento de equivalência (com tabelas de classes válidas/inválidas).
6. Aplicação documentada de análise de valores limite (com tabelas de valores selecionados).
7. Aplicação documentada de cobertura de condições múltiplas (com tabelas de verdade e justificação).
8. Matriz de rastreabilidade completa (ver secção 8), com cobertura bidirecional.
9. Resultados de execução: relatórios de cobertura, capturas de ecrã, ficheiros de exportação.
10. Lista de defeitos detetados, com severidade, passos para reproduzir e estado.
11. Conclusões, limitações e propostas de melhoria.

9.1. Artefactos a Submeter

- Relatório em formato Word ou PDF (este documento expandido).
- Repositório com o código de teste e instruções de execução.
- Coleções Postman/Newman, ficheiros .feature do Karate ou equivalentes, conforme aplicável.
- Relatórios de cobertura de código exportados pela ferramenta utilizada.

10. Considerações Finais

A qualidade do trabalho será avaliada não apenas pela quantidade de testes implementados, mas sobretudo pelo rigor metodológico, pela justificação das opções tomadas e pela capacidade de detetar defeitos relevantes. É expectável que cada grupo descubra, ao longo do trabalho, ambiguidades ou inconsistências na especificação funcional da plataforma GREENHERB; tais descobertas devem ser registadas no relatório como propostas de clarificação ou correção.

Trabalhos cujo plano de testes seja meramente exploratório, sem aplicação documentada das técnicas formais exigidas — particionamento de equivalência, análise de valores limite e cobertura de condições múltiplas — não poderão obter classificação positiva, independentemente da quantidade de casos de teste executados.